SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science & Engineering
CO2 - ASSESSMENT TOOL - 1
Scenario-Based Assessment

---

s

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 1: Scenario-Based Assessment**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	*Design effective test cases using black-box, white-box testing techniques and to control methods flow.*		
Student Name	**M SANDHYA**	**Reg. No.**	**192312421**
Section/Batch	**Slot A / 2023**	**Date**	**31-10-2026**

PATH TESTING STRATEGY FOR A LEGACY DATABASE MIGRATION TOOL

White-Box Evaluation, Independent Path Coverage and Loop Boundary Analysis

1. Introduction and Problem Context

A legacy database migration tool is used to transfer records from an older database structure into a new database system. Before each record is migrated, the tool performs several validation activities such as checking mandatory fields, data types, field lengths, key relationships, duplicate identifiers and referential integrity. The validation logic is implemented using deeply nested loops, where one loop may process tables, another may process records, and inner loops may inspect fields or related values.

The development team reports high statement coverage, but the tool still experiences frequent production crashes described as “infinite loop” failures. This indicates that executing every statement at least once has not demonstrated that the program's control flow is safe for all paths and loop conditions. A statement can execute during a test while a loop's exit condition, repeated execution behaviour, or interaction between nested loops remains untested.

This report evaluates the limitations of the existing white-box approach and proposes a systematic path-testing strategy. The strategy identifies linearly independent paths, tests loop entry and exit behaviour, exercises zero, one, typical and maximum iterations, and uses Cyclomatic Complexity to estimate the minimum basis-path test set. The proposed tests also examine progress variables, termination conditions and nested-loop interactions that can cause non-termination**.**

1.1 Objectives

Evaluate why high statement coverage did not prevent infinite-loop crashes in production.

Model the migration validation logic using a control-flow graph and identify its decision points.

Calculate Cyclomatic Complexity and determine the number of linearly independent basis paths.

Design path tests that exercise independent paths and both outcomes of loop decisions.

Test loop boundary conditions, including zero iterations, one iteration, multiple iterations and maximum permitted iterations.

Detect faulty loop updates, incorrect exit conditions, nested-loop interactions and non-terminating behaviour.

Recommend measurable coverage goals, automation and regression practices for reliable migration.

1.2 Assumed Functional Specification

The scenario does not provide the source code or exact loop limits. Therefore, the following simplified validation model is assumed for designing the test strategy. The actual test values must be adjusted to the implementation and migration requirements.

Rule ID	Assumed migration rule	Expected behaviour
MR-01	Input migration batch is available	Read batch metadata and continue only when the batch is valid.
MR-02	Tables are processed one by one	Each table is visited once and the table index advances.
MR-03	Records within a table are processed	Each record is validated and the record index advances.
MR-04	Fields within a record are checked	Every required field is checked without skipping or repeating indefinitely.
MR-05	Invalid records are reported	Validation errors are logged and processing continues according to policy.
MR-06	Related-key checks are performed	Reference checks terminate after all related values are examined.
MR-07	Loop exit conditions are correct	Every loop has a reachable and deterministic exit condition.
MR-08	Migration limits are respected	Configured maximum records, fields and retry counts are enforced.
MR-09	Completion is reported	The tool exits after all tables and records are processed.

2. Critical Analysis of the Current White-Box Approach

Statement coverage measures whether each executable statement has run at least once. It is useful as a basic measure, but it does not measure whether every decision outcome has been taken or whether loops terminate correctly. In a nested-loop program, a test may execute the loop body and all statements during one successful iteration while never testing the condition that stops the loop, an empty collection, a boundary index, or an error path that changes the loop-control variable.

The reported high statement coverage therefore gives an incomplete picture. The crashes suggest that the existing tests are concentrated on ordinary migration data and successful paths. They may not challenge the loop-control logic with empty tables, a single record, the final record, malformed records, maximum-size batches or relationships that trigger inner-loop repetition.

Aspect	Likely weakness in current testing	Why it can cause production failure
Statement coverage	Statements execute, but decision outcomes are not necessarily both exercised.	A loop condition may always be true in tested cases; the exit branch can remain untested.
Loop boundaries	No explicit zero-, one- or maximum-iteration tests.	Empty collections, final-index errors and off-by-one conditions remain hidden.
Nested loops	Outer and inner loops are not tested independently.	An inner loop may fail to reset its counter or may prevent the outer loop from advancing.
Loop control	Counter updates and termination variables are not verified.	A missing increment, wrong decrement or unchanged condition variable creates non-termination.
Error handling	Malformed records and validation failures are not tested at every loop level.	An exception or continue/retry branch may skip the required update.
Path coverage	No basis-path set is documented.	Some independent routes through the logic may never be executed.
Test data	Only normal-sized migration batches may be used.	Large batches, empty tables and unusual relationships behave differently.
Execution control	No timeout or maximum-iteration safeguard.	A non-terminating test can hang instead of being reported as a failure.

2.1 Probable Root Causes of Infinite Loops

Possible defect	Faulty logic pattern	Test that can expose it
Missing counter increment	while (i < count) { validate(i); }	Run a loop with one or more records and verify i advances to count.
Wrong comparison operator	while (i <= count) with an invalid final index	Test count-1, count and count+1 boundaries.
Inner counter not reset	j is not reset when the next record starts	Use at least two records, each containing multiple fields.
Update skipped on error	continue or exception path bypasses j++	Place an invalid field before the final field and check termination.
Incorrect outer update	Outer table index changes only on successful validation	Use a table containing an invalid record followed by a valid table.
Retry loop without limit	while (status != success) retries forever	Simulate persistent failure and verify retry cap and timeout.
Mutated exit condition	Loop condition depends on a value that is never changed	Use controlled test data and assert a finite iteration count.

3. Control-Flow Model of the Migration Validation Logic

To make path testing measurable, the deeply nested validation routine is represented by a simplified control-flow model. The model contains an outer table loop, a record loop, a field-validation loop and a related-key check. It also includes a validation decision and an error-handling route. Each node represents a process or decision, and each directed edge represents a possible transfer of control.

3.1 Simplified Pseudocode

```
read migration batch
if batch is invalid:
report batch error and exit

table_index = 0
while table_index < table_count:
record_index = 0
while record_index < record_count(table_index):
field_index = 0
while field_index < field_count(record_index):
validate current field
if field is invalid:
record_error = true
field_index = field_index + 1

if record_error:
log record error
else:
check related keys

record_index = record_index + 1
table_index = table_index + 1

report migration summary
exit
```

The pseudocode deliberately shows the required counter updates after the field loop and record loop. In the real implementation, these updates must be checked on every route, including invalid-data, exception, retry and continue paths. If any route bypasses an update that is necessary for termination, the loop may repeat forever.

3.2 Control-Flow Graph Description

Node	Operation / decision	Outgoing paths
1	Start and read migration batch	To node 2
2	Is batch valid?	True → node 3; False → error exit
3	Is table_index less than table_count?	True → node 4; False → node 12
4	Initialise record_index = 0	To node 5
5	Is record_index less than record_count?	True → node 6; False → node 11
6	Initialise field_index = 0	To node 7
7	Is field_index less than field_count?	True → node 8; False → node 10
8	Validate current field	To node 9

9	Record validation result and increment field_index	Return to node 7
10	Log record result / check related keys	To node 11
11	Increment record_index	Return to node 5
12	Increment table_index	Return to node 3
13	Generate summary and exit	Terminal node

The model highlights three nested loop decisions. Each loop has a true route into its body and a false route that exits that loop. The test suite must demonstrate that the true route can repeat and that the false route is eventually reached. It must also prove that the inner loop finishes before the record loop advances, and that the record loop finishes before the table loop advances.

4. Cyclomatic Complexity Calculation

Cyclomatic Complexity, V(G), measures the number of linearly independent paths in a program's control-flow graph. It provides a practical lower bound for the number of basis-path tests needed to exercise the independent control-flow structure. It does not by itself prove that loops terminate; loop-boundary and iteration-count assertions must be added.

4.1 Complexity of the Assumed Control-Flow Model

For the simplified model, the decision nodes are: batch validity, outer table-loop condition, record-loop condition, field-loop condition and field validity. This gives five predicate nodes.

Metric	Calculation	Result
Decision / predicate nodes	D = 5	5
Cyclomatic Complexity	V(G) = D + 1 = 5 + 1	6
Alternative formula	V(G) = E − N + 2P (for one connected component)	6, for the model graph
Independent paths	Minimum basis set size = V(G)	6 paths

The value V(G) = 6 is based on the stated simplified control-flow model. If the actual source code contains additional decisions, such as retry limits, exception handlers, null checks, duplicate detection or multiple related-key conditions, those decision nodes must be added and the complexity recalculated from the actual control-flow graph.

4.2 Basis Set of Independent Paths

Path	Representative route	Purpose / expected outcome
P1	1 → 2(False) → Exit	Invalid batch is rejected before entering any loop.
P2	1 → 2(True) → 3(False) → 13	Valid batch with zero tables exits the outer loop immediately.
P3	1 → 2(True) → 3(True) → 4 → 5(False) → 12 → 3(False) → 13	A table with zero records exits the record loop correctly.

P4	1 → 2(True) → 3(True) → 4 → 5(True) → 6 → 7(False) → 10 → 11 → 5(False) → 12 → 3(False) → 13	A record with zero fields bypasses field validation and completes.
P5	1 → 2(True) → 3(True) → 4 → 5(True) → 6 → 7(True) → 8 → 9 → 7(False) → 10 → 11 → 5(False) → 12 → 3(False) → 13	A field is validated, the field counter advances, and the inner loop exits.
P6	1 → 2(True) → 3(True) → 4 → 5(True) → 6 → 7(True) → 8 → 9 (invalid field outcome) → repeat field loop → 10 → 11 → repeat record loop → 12 → repeat table loop → 13	Invalid data is logged while all nested loops still make progress and terminate.

These paths form an illustrative basis for the simplified model. P6 represents the validation-error route and repeated processing through the nested loops. In practice, basis paths are selected from the actual graph so that each path introduces at least one edge not included in the previously selected paths. The route descriptions should be refined against the source code before execution.

5. Proposed Path Testing Strategy

The proposed strategy combines basis-path testing with targeted loop testing. Basis-path testing ensures that independent control-flow routes are executed, while loop testing concentrates on iteration counts, counter updates and exit conditions. Tests should be deterministic and should record the number of iterations at each nesting level.

5.1 Loop Boundary Conditions

Loop level	Boundary / scenario	Test data	Expected result
Outer table loop	Zero iterations	table_count = 0	No table processing; summary generated; program exits.
Outer table loop	One iteration	table_count = 1	Exactly one table is processed.
Outer table loop	Multiple iterations	table_count = 3	All three tables processed once in order.
Outer table loop	Upper configured limit	Maximum permitted table count	Completes within configured resource and time limits.
Record loop	Zero records	A table containing 0 records	Record loop is skipped; next table is processed.
Record loop	One record	A table containing 1 record	Record validated once; index reaches 1 and loop exits.
Record loop	Many records	A table containing 100 records	All records visited once; no repeats or omissions.
Field loop	Zero fields	A record with 0 optional fields	Field loop exits without invalid index access.

Field loop	One field	A record with 1 field	Field validated once and loop exits.
Field loop	Last field boundary	Field count = n; final valid index = n−1	Final field processed once; no access to index n.
Field loop	Invalid field in middle	Invalid field at index 1 of 3	Error logged; index still advances; remaining fields handled per policy.

5.2 Independent Path Test Cases

TC ID	Path/condition	Test input or setup	Expected result
PT-01	P1 – invalid batch	Missing or unsupported batch metadata	Error reported; no loop is entered.
PT-02	P2 – no tables	Valid batch; zero tables	Normal completion with zero processed tables.
PT-03	P3 – empty table	One table; zero records	Table completes; record loop exits.
PT-04	P4 – empty record	One record; zero fields	No field access; record and table counters advance.
PT-05	P5 – valid field	One table, one record, one valid field	Field validated once; all loops terminate.
PT-06	P6 – invalid field	One record with an invalid field followed by valid fields	Error logged; no infinite loop; remaining processing follows policy.
PT-07	Nested repetition	2 tables × 2 records × 3 fields	Expected 12 field validations; each record and table visited once.
PT-08	Last-index boundary	3 fields indexed 0, 1, 2	Index 2 is processed; index 3 is not accessed.
PT-09	Error path update	Invalid field triggers error/continue branch	Field index still advances; loop exits normally.
PT-10	Outer-loop update	First table has invalid records; second table is valid	Second table is reached; table index is not stuck.
PT-11	Persistent retry failure	Related-key service continuously returns failure	Retry limit or timeout reached; controlled failure reported.
PT-12	Maximum-size batch	Batch at configured maximum size	All records complete within agreed time and memory limits.

5.3 Loop Invariants and Progress Assertions

A loop invariant is a condition that should remain true throughout loop execution. A progress assertion checks that the loop-control state changes so that the exit condition can eventually become false. These checks are especially important for the reported infinite-loop failures.

Outer-loop progress: after each completed table, table_index must increase by exactly one.

Record-loop progress: after each completed record, record_index must increase by exactly one, including records containing validation errors.

Field-loop progress: after each field is checked, field_index must increase by exactly one, including invalid fields and handled exceptions.

Index range: each accessed index must satisfy $0 \leq \text{index} < \text{collection size}$.

Counter reset: record_index must be initialised for each new table, and field_index must be initialised for each new record.

Termination bound: the number of iterations must not exceed the corresponding collection size, except for explicitly bounded retry loops.

Error handling: error logging or a continue statement must not bypass required counter updates.

Completion: the processed table, record and field counts must match the expected counts for the supplied test data.

6. Why Statement Coverage Was Insufficient

Statement coverage answers the question: "Has each executable statement run at least once?" It does not answer: "Have all possible decision outcomes been exercised?" or "Will each loop terminate for every relevant input?" For example, a test containing one table, one record and one field can execute the body of all three loops and achieve statement coverage for the loop statements. However, it may not test an empty collection, multiple iterations, a final-index boundary, or an error route that skips a counter update.

Coverage criterion	What it verifies	Relevance to this scenario
Statement coverage	Every executable statement executes at least once.	Basic execution evidence only; does not prove loop exit behaviour.
Branch coverage	Each decision outcome (true and false) executes at least once.	Confirms loop body and loop-exit branches are both taken.
Condition coverage	Each atomic Boolean condition is true and false at least once.	Useful for compound termination and validation expressions.
Basis-path coverage	Each linearly independent path is executed.	Systematically exercises distinct control-flow routes.
Loop boundary testing	Loop counts and boundary iterations are tested.	Targets zero, one, multiple and maximum iterations.
Stress/endurance testing	Run large workloads for an extended period.	Finds performance-related hangs and resource exhaustion.

Even 100% branch coverage is not a complete proof of termination for arbitrary loop counts. It shows that the decision outcomes were taken, but not necessarily that all relevant iteration boundaries and data-dependent behaviours were tested. Therefore, branch and basis-path coverage must be complemented by loop-boundary tests, iteration limits, watchdog timeouts and progress assertions.

7. Detailed Test Execution and Monitoring Plan

Phase	Activity	Technique / evidence	Acceptance criterion

1. Source review	Inspect every loop condition, counter initialisation and update statement.	Code walkthrough; loop-control checklist.	Every loop has a clear entry, update and exit condition.
2. Graph modelling	Draw control-flow graph and mark all predicates.	Control-flow analysis.	All decisions and loop-back edges are represented.
3. Complexity analysis	Calculate V(G) from the actual graph.	Predicate count and E−N+2P formula.	Basis-path suite contains at least V(G) independent paths.
4. Basis-path execution	Execute independent paths with controlled inputs.	White-box path tests.	Every selected independent path is traversed.
5. Loop boundary execution	Test zero, one, two, typical and maximum iterations.	Loop testing and boundary analysis.	All loops exit at the correct boundary.
6. Fault injection	Inject invalid records, exceptions and persistent service failures.	Negative testing / fault injection.	Error routes do not skip progress updates.
7. Runtime safeguards	Add timeout, maximum iteration and diagnostic logging.	Watchdog and instrumentation.	A suspected hang fails fast with useful diagnostics.
8. Regression automation	Run tests after every code change and migration release.	Automated CI regression.	No previously detected loop defect reappears.

7.1 Instrumentation Recommendations

Log table_index, record_index and field_index at a configurable debug level.

Record start time, end time and iteration count for each loop level.

Add a configurable maximum retry count to all retry loops.

Use a test timeout so a non-terminating test is stopped and reported automatically.

Capture the batch identifier and the last successfully processed table/record when a timeout occurs.

Ensure diagnostic logging does not expose sensitive database values; log identifiers or masked values where appropriate.

8. Cyclomatic Complexity-Based Justification

For the assumed model, V(G) = 6 indicates six linearly independent paths. A minimum basis-path suite of six tests is therefore required to exercise the independent structure of this simplified routine. The proposed twelve path and loop-focused tests go beyond that minimum because one basis path alone cannot adequately represent every important iteration count, nested-loop combination or failure mode.

Cyclomatic Complexity helps estimate structural test effort and identifies routines that may be difficult to maintain. A higher value means more independent routes must be considered. However, a low complexity value does not guarantee safe loop termination: a single while-loop with a faulty

counter update can have low Cyclomatic Complexity and still run forever. This is why complexity metrics must be interpreted together with loop-specific analysis.

Metric	Recommended target	Reason
Basis-path coverage	100% of identified basis paths	Every independent route in the model is executed.
Branch coverage	100% for migration validation decisions	Both loop continuation and loop exit outcomes are exercised.
Loop boundary coverage	100% of identified loop boundaries	Zero, one, multiple and maximum iterations are tested.
Progress assertion pass rate	100%	All loop counters advance or terminate as designed.
Timeout / hang defects	Zero unresolved defects before release	Non-termination is a release-blocking failure.
Regression reproducibility	100% of discovered defects have repeatable tests	Production failures can be prevented from recurring.

9. Challenges and Limitations

Large path space: Real migration code may contain many nested decisions, making exhaustive path testing impractical. Basis-path testing controls the effort but does not cover every possible path combination.

Data-dependent loops: The number of records and fields can vary by database, so representative and maximum-size datasets are required.

External dependencies: Database locks, network delays and remote validation services can create apparent hangs that are not caused by loop counters.

Stateful migration behaviour: A partially completed migration may resume from a checkpoint, introducing additional paths that must be tested.

Concurrency: Parallel migration workers may compete for records or locks, requiring separate concurrency and deadlock testing.

Performance limits: A loop may terminate correctly but still take too long on a large dataset; stress and endurance tests are necessary.

Model assumptions: The complexity value in this report is based on simplified pseudocode. The actual value must be recalculated from the production source code.

10. Conclusion and Future Scope

The infinite-loop crashes in the legacy database migration tool demonstrate that high statement coverage alone is not sufficient to establish control-flow reliability. Statement coverage can confirm that loop-body statements execute, while leaving loop-exit branches, nested-loop interactions and error-handling routes inadequately tested. A missing counter update, incorrect comparison, failure to reset an inner counter or an unbounded retry route can therefore remain undetected until production data activates it.

The recommended solution is to model the validation routine as a control-flow graph, calculate Cyclomatic Complexity, and design a basis set of linearly independent paths. For the assumed five-decision model, V(G) is 6, so at least six independent basis paths are required. The strategy expands

this minimum with targeted tests for zero, one, multiple and maximum loop iterations, invalid records, final-index boundaries and persistent failures. Progress assertions, timeouts, iteration limits and detailed diagnostic logging provide additional protection against non-termination.

In future, the migration tool can be improved by introducing configurable batch limits, resumable checkpoints, automated property-based testing, static analysis for suspicious loop constructs, and performance monitoring for long-running migrations. These measures, combined with branch, basis-path and loop-boundary coverage, will make migration behaviour more predictable, easier to debug and safer for production use.

10.1 References

[1] Glenford J. Myers, Corey Sandler and Tom Badgett, The Art of Software Testing, 3rd ed., John Wiley & Sons, 2011.

[2] Boris Beizer, Software Testing Techniques, 2nd ed., Van Nostrand Reinhold, 1990.

[3] Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed., McGraw-Hill, 2020.

[4] Paul C. Jorgensen, Software Testing: A Craftsman's Approach, 4th ed., CRC Press, 2014.

[5] IEEE, IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008, 2008.

[6] ISO/IEC/IEEE, Systems and Software Engineering – Software Testing, ISO/IEC/IEEE 29119 series.

[7] ISTQB, Certified Tester Foundation Level Syllabus, International Software Testing Qualifications Board.

[8] T. J. McCabe, "A Complexity Measure," IEEE Transactions on Software Engineering, vol. SE-2, no. 4, pp. 308–320, 1976.

Code of Conduct:

I, M Sandhya(192312421), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Sandhya**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements.	Clearly understands the scenario with minor gaps.	Basic understanding; some aspects of the scenario are unclear.	Poor understanding or misinterpretation of the scenario.	
Application of Concepts	30	Applies appropriate concepts effectively to	Applies relevant concepts correctly	Applies basic concepts but with limited accuracy.	Fails to apply appropriate concepts.	

		solve complex real-world problems.	with minor errors.		
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification.	Good analysis with reasonable solutions.	Limited analysis; solutions lack justification.	Poor or no analysis; incorrect decisions.
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach.	Logical and structured solution with minor gaps.	Basic solution with limited structure or feasibility.	Unclear or incorrect solution approach.
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning.	Clear explanation with some justification.	Limited clarity and weak justification.	Poorly explained with no justification.